

CS3523: Operating Systems II (Spring 2026)
IIT Hyderabad, India

Mid-Semester Examination Answer Key

25 February 2026, 04:00 to 06:00 PM IST

Instructions:

1. Write the Roll No, Name and Department on the first page of the answer booklet. On all subsequent pages, write only the Roll No.
2. Read all questions carefully before beginning to answer.
3. All answers must be written in the space provided in the question paper itself. Additional sheets will not be evaluated unless explicitly permitted.
4. One additional page has been provided at the end of the booklet for rough work. No marks will be awarded for anything written on the rough work page.
5. Mobile phones, smart devices, and any unauthorized electronic equipment are strictly prohibited.

Roll No: _____

Name: _____

Department: _____

Marks Distribution

	Max Marks	Obtained Marks
Q1	20	
Q2	15	
Q3	15	
Q4	30	
Total	80	

Q1. Scheduling [2 Marks Each]

Three jobs arrive to a single-processor system in the following order: job A arrives at time 0 and requires 8 seconds of CPU time, job B arrives immediately after A and requires 12 seconds of CPU time, and job C arrives immediately after B and requires 8 seconds of CPU time.

Unless otherwise stated, context switch overhead is zero. For round-robin scheduling, if job X arrives immediately before job Y, X will be scheduled before Y.

(a) Under a Shortest-Job-First (SJF) policy, at what time does job B complete execution?

Order of completion: A, C, B. B finishes last, after 28 seconds.

(b) Under a Longest-Job-First (LJF) policy, at what time does job B complete execution?

B, A, C. B finishes first, after 12 seconds.

(c) Under a Round-Robin (RR) policy with a time quantum of 1 second, at what time does job A complete?

A needs to run eight times to finish (8 seconds @ 1 second/slice).

ABC	ABC	ABC	ABC	ABC	ABC	ABC	ABC
012	345	678	901	234	567	890	123
---	---	---	-11	111	111	112	222

The first A starts at 0 and ends at 1. Then starts at 3 and ends at 4. Then 6 to 7, 9-10, 12-13, 15-16, 18-19, and finally, starts at 21 and completes its last time slice at 22. Thus, 22.

(d) Under a RR policy with a time quantum of 1 second, at what time does job B complete?

This is easy. B runs the longest and thus finishes last. We know the total time for all workloads is 28. Thus, B finishes at 28.

(e) Under a RR policy with an unknown time quantum less than or equal to 2 seconds, at what time does job B complete?

With any small time slice, B finishes last again, and thus at 28 again.

(f) Under a RR policy with an unknown time quantum, for what values of the time quantum does job B complete before job C?

There are two interesting cases. When the time slice is greater than or equal to 12, B will finish in its first time slice and thus finish before C. The other case arises when the time slice is greater than or equal to 6 seconds but less than 8 seconds. In this case, B will finish in two slices (as will C), but B will finish first.

In practice, scheduling policies such as SJF depend on predicted runtimes. In this system, each job supplies an estimated runtime to the Operating System (OS). However, user estimates are imperfect. If a job reports an estimated runtime of N seconds, the actual runtime may lie anywhere between $N-4$ and $N+4$ seconds. The OS, however, trusts the reported value exactly when making scheduling decisions.

(g) Under a SJF policy, what set of reported estimates would cause the scheduler to make the most unfavourable ordering decision with respect to minimising average response time?

Anything that makes B run first. Thus, any estimate when A's and C's estimates are each X and B's estimate is Y and $X > Y$. This is possible because the estimates can only be off by 4 seconds in either direction but the run times of B, (A,C) are only 4 seconds apart.

```

|-----A-----|
|-----C-----|
|-----B-----|

```

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20

(h) Under this worst-case mis-estimation, what would the average response time be?

B's response time will be 0 (it ran first). A's would be 12 (after B finishes), and C's would be 20 (after A finishes). Thus, $(0 + 12 + 20)/3$, or 10.67.

(i) If LJF policy were used instead, what would the average response time be?

If we used LJF with our mis-estimates, C would run (0), then A (8), and then B (16). Thus $24/3 = 8$.

(j) Suppose the runtime of job B may be adjusted arbitrarily. Determine the value to which B's runtime should be changed so that SJF always yields the minimum possible average response time, regardless of estimation error within the allowed bounds.

Two cases: (a) 8 seconds apart, because the worst mis-estimate is 4 seconds in one direction and 4 seconds in the other. (b) 8 seconds, i.e., it is made equal to A's and C's runtime.

Q2. Segmentation

Segmentation is a memory-virtualisation mechanism that relies on base and bounds registers to translate virtual addresses into physical addresses while enforcing protection.

(a) What type of address is stored in a base register: physical or virtual? **[1 Mark]**

Physical

(b) How many segments should the hardware support? Justify in terms of typical program structure. **[2 Marks]**

At least two (say one for code+heap, the other for the stack) gives the system the ability to support sparse address spaces. Three could also be useful, as adding a separate one for code allows code to be shared. More than that could be useful but perhaps beyond the scope of the exam.

Now consider the following system configuration: the virtual address space is 1KB while the physical memory is 8KB. The hardware supports two segments; segment-0 base register value is 1KB and bounds (size) is 400B and it grows upward. Whereas segment-1 base register value is 4KB and bounds (size) is also 400B but it grows backward (negative direction).

Consider the following program fragment:

```
void *ptr = 20;

while (ptr <= 1024) {

    int x = (int *) *ptr; // LINE 1: read what is at address 'ptr'

    ptr = ptr + 20; // LINE 2: increment 'ptr' to a new address

}
```

(c) What virtual addresses are generated at LINE 1 by dereferencing `ptr`? (Assume the program runs to completion; ignore instruction fetches, store into `x`, etc. Just listing the addresses is enough). [3 Marks]

Starts at virtual address 20; increments by 20 until 1020 (assuming no crash, etc.)

(d) After how many iterations will the program crash due to a segmentation violation? Justify. [3 Marks]

Access to virtual addresses 20, 40, 60, 80, 100, 120, 140, 160, 180, 200, 220, 240, 260, 280, 300, 320, 340, 360, 380 all are fine (19 iterations). Access to 400 exceeds the bounds of segment 0 and thus will cause an exception (we know that the virtual addresses are in the 0th segment because the top bit of these addresses are 0; virtual addresses between 512-1023 have the top bit set to 1 and thus are in the 1st segment).

(e) What physical addresses are generated by dereferencing `ptr` before the program crashes? [3 Marks]

The above virtual addresses must be added to the seg 0 base register (value 1024) to get physical addresses. Thus, 1044, ..., 1404 will be generated before crashing.

(f) What legal physical address range could have been accessed by dereferencing `ptr`, if the program had been written to avoid violating segment bounds? [3 Marks]

If the program skipped the loop iterations that would cause a crash, it would skip the middle part of the address space which is not mapped and has no legal addresses. The last 400 bytes of the AS are also legal (below 1024 starting at 624), and hence those could have been accessed legally. For example, if 1020 was accessed, it would be legal and translate to 4KB minus 4 or 4092 (remember the stack grows the other way). The first legal address in this range would be 640 which translates to 3712. Thus, every virtual address between 3712 and 4092 (inclusive) could have been legally accessed, if the loop skipped over 400 to 620 (which would all fault).

Q3. Paging

Assume a system with the following characteristics: 32-bit virtual address space, 2 KB page size, and each Page Table Entry (PTE) is 4 bytes.

(a) If a linear page table is used, what is the total memory required to store the page table for one process? Show the calculation clearly. **[2 Marks]**

11 bits (for a 2KB page, you need 11 bits to address each byte).

That leaves 21 bits for the VPN (32 bits total - 11 bits of offset).

$2^{21} \times 4$ because each PTE is 4 bytes in size (as stated in the question). 2^{20} is 1MB; hence 2^{21} is 2MB; hence the total answer is 8MB for the entire page table.

Linear page tables can consume significant memory. To reduce this overhead, a multi-level page table may be used. Assume the system implements a two-level page table using a page directory, and each Page Directory Entry (PDE) is 4 bytes.

(b) Compute offset bits, PT index bits, PD index bits. How many entries per page table? Show all calculations. **[3 Marks]**

Offset is 11 bits.

We know a PTE is 4 bytes in size. Hence, 512 entries fit into one 2KB page.

We need 9 bits to tell us which entry we are referring to ($2^9 = 512$).

Given that the VPN is 21 bits, and subtracting 9 for the PT index bits, that leaves us 12 bits for the PD index bits.

(c) How much memory is required to store the page directory? **[2 Marks]**

With 4 bytes PDEs, we get 2^{12} entries times 4 bytes/entry or 16KB

Now consider the following three independent memory allocation scenarios. In each case, determine the total memory consumed by the two-level page table, and by the linear page table.

(d) Code is located at address 0 and there are 150 4-byte instructions. The heap starts at page 1 and uses 5 total pages. The stack starts at the other end of the address space, grows backward, and uses 3 total pages.

Multi-level page table size? **[2 Marks]**

Each chunk of the page table covers 512 consecutive pages; hence the code pages and heap in this example fit onto the same chunk of the page table. The stack, at the other end of the address space, needs one too. Hence 2 pages (2KB each) plus the page directory (16KB), or 20KB.

Linear page table size? **[1 Mark]**

The linear page table is always the same regardless of the usage of pages in the address space. Hence, 8MB.

(e) Code is located at address 0 and there are 200 4-byte instructions. The heap starts at page 1 and uses 1001 total pages. The stack starts at the other end, grows backwards, and uses 500 total pages.

Multi-level page table size? **[2 Marks]**

Here we need two pages of the page table to translate the first 1001 pages of the address space (two pages of the page table covers 1024 pages) and another one page to translate the last 500 pages of the stack. Hence, 3 pages (2KB each) plus the page directory (16KB) gets us to 22KB.

Linear page table size? **[1 Mark]**

8MB

(f) The entire address space (every page) is used by the process.

Multi-level page table size? **[1 Mark]**

If all pages are used, you get the full linear page table (8MB) plus the page directory (16KB).

Linear page table size? **[1 Mark]**

8MB

Q4. Virtual Memory [3 Marks Each]

Assume the following system configuration: 32-bit virtual addresses, 4KB page size, 1024 physical page frames, a 32-entry TLB, linear page tables, and Least Recently Used (LRU) replacement policy for both TLB and physical memory. Before the execution begins, the following *initialisation code* is run once:

```
void *orig = malloc(NUM_PAGES * PAGE_SIZE); // allocate NUM_PAGES*PAGE_SIZE bytes

void *ptr = orig;

for (i = 0; i < NUM_PAGES; i++) {

    (int *) *ptr = 0; // init first value on each page

    ptr += PAGE_SIZE;

}
```

This code simply allocates (NUM_PAGES * PAGE_SIZE) number of bytes and then sets the first integer on each page to 0, where PAGE_SIZE is 4KB (as above) and NUM_PAGES is a constant (defined below). Assume malloc() returns page-aligned data in this example. Now, the *test code* is the following:

```
ptr = orig;

for (i = 0; i < NUM_PAGES; i++) {

    int x = (int *) *ptr; // load value pointed to by ptr

    ptr += PAGE_SIZE;

}
```

For the following questions, only consider memory references to the allocated region via `ptr` (ignore instruction fetches, store into `x`, etc.). How many TLB hits, TLB misses, and Page Faults occur during the test code when:

	TLB hits	TLB misses	Page Faults
(a) <code>NUM_PAGES</code> is 16?	16	0	0
(b) <code>NUM_PAGES</code> is 32?	32	0	0
(c) <code>NUM_PAGES</code> is 2048?	0	2048	2048

The workload just loops over some pages accessing each one once. Thus, the second time through the loop with a small number of pages (less than the TLB size, for example) just yields TLB hits. With a large number of pages, and LRU replacement, nothing we want is ever in the cache or TLB, hence all misses/page faults.

Assume a memory reference takes roughly time M and that a disk access takes time D . How long does the test code take to run (approximately), in terms of M and D , when:

	TLB hits	TLB misses	Page Faults
(d) <code>NUM_PAGES</code> is 16?	16M		
(e) <code>NUM_PAGES</code> is 32?	32M		
(f) <code>NUM_PAGES</code> is 2048?		2048 X 2M	2048D

A hit means a load to memory which we assume costs M and hence the TLB hits just cost M . On TLB misses we need to consult memory twice, once for the page table (minimally) and once for the actual memory load, hence $2M$ per TLB miss. Finally, each page fault costs us a disk access.

Now assume that the various replacement policies in the system is changed to Most Recently Used (MRU). Given this change, how long does the test code take to run (approximately), in terms of M and D , when:

	TLB hits	TLB misses	Page Faults
(g) <code>NUM_PAGES</code> is 16?	16M		
(h) <code>NUM_PAGES</code> is 32?	32M		
(i) <code>NUM_PAGES</code> is 2048?	32M	2016 X 2M	1024D

With MRU, the first 31 accesses get lodged in the TLB, and the first 1023 pages get lodged in memory; subsequent accesses simply push the most-recently used TLB entry/page out. The next time through

the loop, the first 31 accesses are thus hits, and the first 1023 page accesses are in memory; everything else are misses until the last page, which also is a hit (do a small MRU example if this doesn't make sense). Thus, 32 TLB hits (the rest misses); 1024 page faults (the rest in memory).

Suppose the test program is run on an unfamiliar machine with unknown TLB size and unknown physical memory size.

(j) Describe how the test program could be used to experimentally determine the capacity of TLB and physical memory.

Just looking for something simple here. You could basically time how long an iteration of the loop lasts. If it lasts something like NUM PAGES times memory access time, those are TLB hits; if it lasts twice that, TLB misses, and thus you can know how big the TLB is. One more big jump occurs when NUM PAGES is finally bigger than memory and causes lots of (very slow) disk accesses.

Rough Work (will not be evaluated)